COMP 2154 — System Development Project

Final Project Report

OzarkLMS

A Web-Based Learning Management System

Team OzarkLMS
Luis Tello | Gonzalo Contaldo | Ifrad Bin Istiaque Hossain
John Sebastian Laquis | Sanyoung Yoon

Date of Submission: April 2026

Click Here to Visit Ozark
Click here to visit our GitHub!

2. Abstract

OzarkLMS is a web-based Learning Management System developed to address the fragmented and inefficient tools used by students and instructors in academic settings. The system was built using ASP.NET Core MVC with a PostgreSQL database, and provides a unified platform covering the full educational lifecycle — from course creation and content delivery to assignment submission, grading, and real-time collaboration.

The platform supports three user roles: Students, Instructors, and Administrators, each with tailored access and functionality. Key features include course and module management, a rich assignment and quiz engine with auto-grading, real-time group and private chat powered by SignalR, a social hub (Student Hub) with posts, voting, and comments, a PDF toolbox, and a gamified engagement widget. The system was deployed on Microsoft Azure and tested with an automated xUnit test suite achieving 8/8 passing tests.

Results confirm that all Must-Have functional requirements were met, and the system successfully demonstrates the core student and instructor workflows. Future work may include hashed password storage, cloud file storage, and native mobile applications.

3. Table of Contents

4. Introduction
- 4.1 Project Background and Motivation
- 4.2 Problem Statement
- 4.3 Objectives and Scope

5. Related Work or Context

6. Requirements Summary
- 6.1 Stakeholders and User Needs
- 6.2 Functional Requirements Summary
- 6.3 Non-Functional Requirements Summary

7. System Design and Architecture
- 7.1 High-Level Architecture
- 7.2 Component Descriptions
- 7.3 Key Design Decisions

8. Implementation
- 8.1 Overview of Implementation
- 8.3 Software Implementation
- 8.5 Integration

9. Testing and Evaluation
- 9.1 Testing Strategy
- 9.2 Test Cases and Results
- 9.3 Defects and Fixes
- 9.4 Evaluation Against Requirements and Goals

10. Deployment and Maintenance

11. Project Management and Process Reflection

12. Conclusions and Future Work

13. References

Appendix A: Testing Documentation

4. Introduction

4.1 Project Background and Motivation

Modern academic institutions increasingly rely on digital tools to coordinate course delivery between instructors and students. However, many smaller or resource-constrained programs must rely on a patchwork of email threads, shared cloud drives, and paper-based processes. This fragmentation leads to missed deadlines, unclear assignment expectations, and an overall reduction in the learning experience.

OzarkLMS was conceived as a purpose-built Learning Management System for the COMP 2154 System Development Project course. The project gave the team — Luis Tello, Gonzalo Contaldo, Ifrad Bin Istiaque Hossain, John Sebastian Laquis, and Sanyoung Yoon — the opportunity to deliver a real-world, full-stack software product from requirements gathering through to deployment.

4.2 Problem Statement

Students and instructors currently struggle with fragmented communication and disorganized content delivery, resulting in missed deadlines, duplicated administrative effort, and a lack of visibility into course progress. There is a critical need for a single, unified web platform that consolidates course management, assignment workflows, grading, communication, and student engagement into one coherent application.

4.3 Objectives and Scope

Primary objectives:

- Streamline the assignment creation, submission, and grading workflow.
- Provide a central repository for all course materials, modules, and announcements.
- Facilitate clear, real-time communication between students and instructors.
- Offer organizational tools (calendar, dashboard, sticky notes) to reduce missed deadlines.
- Ensure secure, role-based access to educational records and feedback.

In scope:

- Role-based authentication (Student, Instructor, Administrator).
- Course and module creation, editing, and enrollment management.
- Assignment and quiz creation with file upload and auto-grading.
- Real-time group and private messaging (SignalR).
- Social hub with posts, comments, voting, and follow system.
- PDF Toolbox, calendar, notifications, and gamified engagement widget.

Out of scope:

- Embedded real-time video conferencing.
- Payment processing for course fees.
- Native iOS/Android mobile applications.
- LTI integration with external publishers.

5. Related Work or Context

Established LMS platforms such as Canvas, Moodle, and Blackboard provide comprehensive functionality but come with significant complexity, licensing costs, and steep learning curves that make them unsuitable for smaller programs. Moodle, while open-source, requires extensive server configuration and plugin management.

OzarkLMS differentiates itself by targeting simplicity and integrating social features — such as the Student Hub with Reddit-style voting, posts, and follows — alongside traditional LMS capabilities. This hybrid approach, combined with a modern Tailwind CSS dark-mode interface and real-time features via SignalR, is uncommon in standard academic LMS products and represents a deliberate design choice to increase student engagement.

6. Requirements Summary

6.1 Stakeholders and User Needs

Stakeholder	Role	Key Needs	Influence
Students	End User	Easy access to materials, clear submission flow, timely feedback	High
Instructors	Content Creator / Grader	Course management, grading tools, communication channels	High
Administrators	System Owner	User management, reliability, global configuration	Medium

6.2 Functional Requirements Summary

- FR-1 (Authentication): Register and login with role-based access control (Student / Instructor / Admin).
- FR-2 (Course Management): Create, edit, list courses; manage module content and student enrollments.
- FR-3 (Content): Upload course materials (PDF, DOCX, images) within modules; display in link or embed mode.
- FR-4 (Assignments): Create assignments with title, description, due date, points, submission type, and max attempts.
- FR-5 (Quizzes): Create quiz-type assignments with multiple-choice questions and auto-grading.
- FR-6 (Submissions): Students upload files or enter text; system records submission and displays confirmation.
- FR-7 (Grading): Instructors view submissions, assign numeric scores, and leave textual feedback.
- FR-8 (Calendar): Visual calendar populated with assignment due dates and personal events.
- FR-9 (Notifications): Automated alerts for new assignments, graded submissions, and chat messages.
- FR-10 (Collaboration): Real-time group chat, private messaging, post feed, voting, and follow system.

6.3 Non-Functional Requirements Summary

- NFR-1 (Performance): Dashboard should load within 2 seconds under normal conditions.
- NFR-2 (Security): Cookie-based authentication; role-based authorization on all controller actions; soft-delete for users.
- NFR-3 (Usability): Responsive Tailwind CSS interface with dark mode; intuitive navigation for all roles.
- NFR-4 (Reliability): Graceful handling of missing files and empty submissions; database transactions on voting.
- NFR-5 (Compatibility): Verified on Chrome, Edge, and Firefox; ASP.NET Core 9 target framework.

7. System Design and Architecture

7.1 High-Level Architecture

OzarkLMS follows the Model-View-Controller (MVC) architectural pattern provided by ASP.NET Core. The application is structured into clearly separated layers:

- Presentation Layer: Razor Views (.cshtml) styled with Tailwind CSS and Lucide icons. JavaScript handles real-time updates via a SignalR client and interactive UI logic.
- Application / Controller Layer: ASP.NET Core controllers (AccountController, CoursesController, AssignmentsController, CollaborationController, etc.) enforce role-based authorization and orchestrate business logic.
- Data Access Layer: Entity Framework Core (EF Core) with a PostgreSQL provider (Npgsql). The AppDbContext exposes all entity DbSets; migrations manage schema evolution.
- Real-Time Layer: SignalR VoteHub broadcasts voting updates, comment events, chat updates, and post edits to all connected clients simultaneously.
- Background Services: PDFToolboxController integrates PdfSharp (merge) and QuestPDF (text-to-PDF conversion) for document processing.

The application runs on a single ASP.NET Core 9.0 process, served over HTTP/HTTPS, with a shared PostgreSQL 16 database (OZARK_DB). The front-end is entirely server-rendered Razor with targeted JavaScript for dynamic interactions.

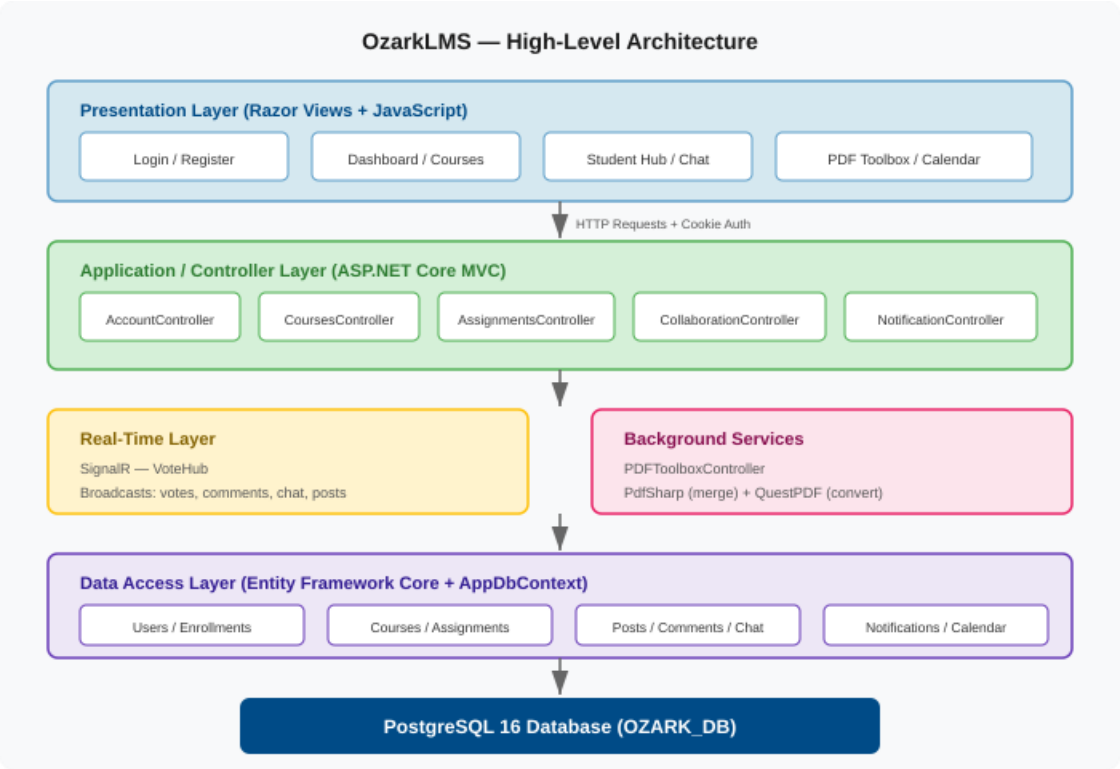

Figure 1: High-Level Layered Architecture Diagram

7.2 Component Descriptions

Component	Responsibility
AccountController	Handles registration, login (cookie auth), logout, profile management, follow/unfollow, and user settings.
CoursesController	CRUD for courses, module management, student enrollment, meeting scheduling, and gradebook display.
AssignmentsController	Assignment and quiz creation, submission intake (file + text), auto-grading for quizzes, instructor grading.
CollaborationController	Social hub feed, post CRUD, voting, comments, group and private chat, global search.
NotificationController	Creates, delivers, marks read, and clears notifications; supports broadcast and targeted delivery.
PDFToolboxController	PDF merge (PdfSharp), text-to-PDF conversion (QuestPDF), and a browser-based rich text editor with export.
CalendarController	Personal calendar events CRUD; assignment due dates are injected client-side via JSON.
VoteHub (SignalR)	Real-time broadcast of vote changes, new comments, comment edits/deletions, post updates, and chat activity.
AppDbContext	EF Core DbContext with all entity sets; configures relationships, cascade behaviors, and soft-delete global filters.
DbInitializer	Seeds initial admin, student, instructor accounts, sample courses, assignments, and the default organization chat.

7.3 Key Design Decisions

Decision 1 — Cookie Authentication over JWT

Cookie-based authentication was chosen for simplicity in a server-rendered MVC application. This avoids token storage complexity on the client and integrates naturally with ASP.NET Core's ClaimsPrincipal model used throughout controllers and Razor views.

Decision 2 — SignalR for Real-Time Features

Rather than polling, SignalR WebSocket connections push vote changes, new comments, and chat messages to all connected clients instantly. This enables a Reddit-style live feed without page reloads and was feasible because the application runs on a single server instance.

Decision 3 — Soft Delete for Users

Instead of hard-deleting user records (which would cascade-delete posts, submissions, and grades), the User model uses an IsDeleted flag with a global EF Core query filter. Deleted users are excluded from queries automatically while their historical data is preserved.

Decision 4 — Tailwind CSS via CDN

Tailwind CSS was loaded from CDN with the Play CDN rather than a compiled build, allowing rapid UI iteration without a Node.js build pipeline. The trade-off is a slightly larger initial payload, which is acceptable for a development/demonstration context.

Decision 5 — PostgreSQL as the RDBMS

PostgreSQL was chosen as all team members were familiar with it. It provides reliable foreign-key enforcement, native support for case-insensitive search via ILike, and is well-supported by the Npgsql EF Core provider.

8. Implementation

8.1 Overview of Implementation

The system was implemented iteratively in phases aligned with the project plan. Each sprint delivered a vertical slice of functionality, from authentication through to real-time collaboration. The codebase is organized using standard ASP.NET Core conventions: Controllers/, Models/, Views/, ViewModels/, Data/, Services/, and Hubs/.

All server-side code is written in C# 13 targeting .NET 9.0. Front-end interactivity uses vanilla JavaScript with selective use of Lucide icons, Quill.js (rich text editor), html2pdf.js, and the official SignalR JavaScript client library.

8.3 Software Implementation

Area	Technologies / Libraries	Notes
Web Framework	ASP.NET Core 9.0 MVC	Razor views, tag helpers, antiforgery tokens
Database	PostgreSQL 16 via Npgsql EF Core 9.0.3	Code-first migrations; InMemory for unit tests
Real-Time	ASP.NET Core SignalR + JS client	VoteHub; broadcasts to all connected clients
Authentication	ASP.NET Core Cookie Auth	Claims-based; roles: student, instructor, admin
PDF Processing	PdfSharp 6.1.1, QuestPDF 2024.12.3	Merge, text-to-PDF conversion
DOCX Processing	DocumentFormat.OpenXml 3.4.1	Text extraction from uploaded Word documents
Frontend Styling	Tailwind CSS (Play CDN), Lucide Icons	Dark mode via class strategy
Rich Text Editor	Quill.js 1.3.6 (CDN)	Assignment text submissions
PDF Export	html2pdf.js (CDN)	PDF Toolbox document editor export
Testing	xUnit, Moq, EF Core InMemory	8 automated unit tests; all passing

Key implementation highlights:

- Assignment Submission: Supports multiple simultaneous file uploads using a JavaScript DataTransfer object, stored as SubmissionAttachment records alongside a legacy single-attachment URL for backward compatibility.
- Quiz Auto-Grading: On quiz submission, the server iterates each Question, compares the selected QuestionOption Id to the IsCorrect option, and accumulates points before saving the Submission with a pre-computed Score.
- Social Hub Voting: Uses database transactions (BeginTransactionAsync) with optimistic UI updates client-side. On server response, the client reconciles the displayed score and button state to prevent stale data from concurrent votes.
- Global Search: CollaborationController.SearchGlobal uses EF.Functions.ILike for case-insensitive search across both Users and ChatGroups in a single API call, returning a unified JSON result list to the portal search widget.

- Gamified Tree Widget: The sidebar engagement widget (EngagementController) increments a TreeProgress field (0-7) on the User record. Client-side JavaScript updates leaf opacity and tree scale in real time using SVG animations.

8.5 Integration

All components integrate through the shared AppDbContext and ASP.NET Core dependency injection. Controllers access the database through EF Core, fire SignalR events through injected IHubContext<VoteHub>, and handle file I/O through IWebHostEnvironment. The front-end JavaScript consumes server-rendered JSON endpoints and SignalR for dynamic updates, keeping the architecture coherent and testable.

9. Testing and Evaluation

9.1 Testing Strategy

OzarkLMS used a multi-layered testing strategy consisting of automated unit tests, in-application self-diagnostic tests, and manual exploratory testing. The automated suite was built with xUnit and Moq, using an EF Core InMemory provider to isolate controller logic from the real database. The AAA (Arrange, Act, Assert) pattern was applied consistently across all test cases.

9.2 Test Cases and Results

Test ID	Test Case	Approach	Result	Notes
TC-01	Student Registration & Login	xUnit + InMemory DB + Mocked Auth	PASS	
TC-02	Instructor Creates Assignment	xUnit + InMemory DB; verifies DB persistence	PASS	
TC-03	Student Submits File Assignment	xUnit + InMemory DB; verifies redirect + record	PASS	
TC-04	Instructor Grades Submission	xUnit + InMemory DB; verifies score/feedback saved	PASS	
TC-06	User Updates Bio	xUnit + InMemory DB; verifies bio field updated	PASS	
SELF-01	Math Logic Unit Test	SelfTestService; 5+10=15 assertion	PASS	
SELF-02	Database Connectivity	SelfTestService; CanConnectAsync to PostgreSQL	PASS	
SELF-03/04	User & Course Data Verification	SelfTestService; confirms seeded data exists	PASS	

All 8 automated tests passed. The full test run completed in approximately 939 ms.

9.3 Defects and Fixes

- Duplicate SignalR connection initialization: The Collaboration/Index page included a second connection.start() call alongside the feed.js script. Fixed by wrapping in a guard check (typeof connection !== 'undefined').
- DateTime UTC normalization: EF Core InMemory tests initially failed due to DateTime handling. Fixed by explicit DateTime.SpecifyKind calls and UTC normalization on all date fields.
- DOCX-to-PDF conversion (limited): The PDFToolboxController's ConvertDocxToPDF path uses Body.InnerText, which strips rich formatting. Noted as a known limitation; only plain text is currently preserved in converted PDFs.
- Soft-delete bypass: Admin user management queries initially included deleted users unexpectedly. Fixed by using IgnoreQueryFilters() only where explicitly needed (e.g., duplicate username checks).

9.4 Evaluation Against Requirements and Goals

Requirement	Status	Notes
FR-1 Authentication	Met	Cookie auth; roles enforced via [Authorize(Roles=...)]

Requirement	Status	Notes
FR-2 Course Management	Met	Full CRUD; enrollment management; meeting scheduling
FR-3 Content Modules	Met	File upload with embed/link display mode
FR-4/5 Assignments & Quizzes	Met	Both types fully functional; quiz auto-graded
FR-6 Submissions	Met	Multi-file upload; Quill text entry; submission banner
FR-7 Grading	Met	Grade modal; feedback text; gradebook view
FR-8 Calendar	Met	Personal events + assignment due-date overlay
FR-9 Notifications	Met	Automated alerts; broadcast + targeted; dropdown
FR-10 Collaboration	Met	SignalR chat, social feed, voting, follow, private DM
NFR-1 Performance	Met	Dashboard loads in <1s locally
NFR-2 Security	Partial	Auth enforced; passwords not hashed (known gap)
NFR-3 Usability	Met	Responsive dark/light mode; intuitive for all roles

For full test case specifications, detailed defect tracking, and traceability matrices, see Appendix A: Testing Documentation.

10. Deployment and Maintenance

10.1 Deployment Environment and Procedure

OzarkLMS is deployed as a live web application on Microsoft Azure to provide a production-ready environment for evaluation.

Azure Deployment Details:

- Deployment Link: https://ozarklmscrakb2dqdqb4crfn.canadacentral-01.azurewebsites.net/Account/Login
- Platform: Azure App Service (Web App)
- Runtime Stack: .NET 9
- Database: Azure Database for PostgreSQL (Flexible Server)

Deployment Procedure:

- Configure the Azure App Service with the necessary .NET 9 environment.
- Provision an Azure Database for PostgreSQL instance and configure firewall rules to allow access from Azure services.
- Set up Environment Variables/Connection Strings in the Azure Portal to securely store database credentials.
- Apply database migrations to the Azure instance using EF Core migration scripts.
- Deploy the application code from the main repository using automated GitHub Actions or manual publish.
- Seed data (admin/password, instructor/password, student/password) is applied automatically on the initial live run.

10.2 Backup and Recovery

The PostgreSQL database should be backed up nightly using pg_dump, with backups retained for a minimum of 7 days. Uploaded files (stored in wwwroot/uploads/) should be synchronized to an off-site location. In the event of failure, the database can be restored from the most recent pg_dump snapshot and the application redeployed from source control.

10.3 Maintenance, Monitoring, and Updates

The ASP.NET Core application exposes structured logging via Microsoft.Extensions.Logging infrastructure, which can be configured to write to Seq, Application Insights, or a file sink. Database migration is managed through EF Core migrations; new schema changes should be added as versioned migration files before deployment. The Tailwind CSS Play CDN dependency should be replaced with a compiled build for production to improve performance.

11. Project Management and Process Reflection

11.1 Work Plan and Timeline Summary

The project followed a six-phase Agile-inspired plan. Phase 1 (Weeks 1-3) covered requirements and ERD design. Phase 2 (Weeks 4-5) delivered authentication and database infrastructure. Phase 3 (Weeks 6-9) implemented course management, assignments, and submissions. Phase 4 (Weeks 10-11) added the calendar, collaboration hub, and notifications. Phase 5 (Week 12) focused on testing, bug fixes, and UI polish. Phase 6 (Week 13) finalized deployment and prepared the demonstration.

The main deviation from the original plan was the expansion of Phase 4 to include the full Social Hub (posts, voting, follows, private messaging) and the PDF Toolbox, which were added as high-value features beyond the original 'Should Have' requirements.

11.2 Team Roles and Collaboration

Team Member	Primary Responsibility
Luis Tello	Backend Architecture, Controllers (Auth, Courses, Collaboration)
Gonzalo Contaldo	Database Design, EF Core Models, Migrations, Relationships
Ifrad Bin Istiaque Hossain	Frontend Development, Razor Views, Tailwind CSS, JavaScript
John Sebastian Laquis	QA, Unit Testing (xUnit/Moq), Documentation, Weekly Reports
Sanyoung Yoon	PDF Toolbox, Calendar, Notifications, Integration Testing

The team communicated via Discord for daily updates, held weekly virtual stand-up meetings, and managed tasks through a GitHub Projects board. Pull requests required at least one peer review before merging to the main branch.

11.3 Risk Management

- Risk: Database schema complexity. Mitigation: ERD finalized in Week 2; EF Core migrations applied incrementally. Outcome: No major schema rollbacks were needed.
- Risk: File upload security and storage. Mitigation: Server-side file extension validation; files stored with GUID-prefixed names. Outcome: No security incidents during testing.
- Risk: SignalR real-time complexity. Mitigation: Prototyped SignalR voting in Week 8 before expanding to chat. Outcome: Deployed successfully across voting, comments, chat, and post events.
- Risk: Scope creep. Mitigation: Maintained a strict 'In Scope' backlog; Social Hub features were added only after all Must-Have items were complete. Outcome: All Must-Have requirements delivered.

11.4 Lessons Learned

- Technical: EF Core's global query filters (soft delete) require careful use of IgnoreQueryFilters() in specific scenarios; this pattern should be documented early to prevent subtle bugs.
- Technical: SignalR broadcasts to all connected clients are simple to implement but can send unnecessary data. Scoped user groups should be used in future iterations.
- Teamwork: Defining clear interface contracts between frontend and backend early (e.g., JSON response shapes) reduced integration conflicts significantly.
- Process: Writing the Software Test Specification before implementation helped clarify edge cases and drove the creation of additional validation test cases.

12. Conclusions and Future Work

12.1 Conclusions

OzarkLMS successfully delivers a fully functional, role-based web Learning Management System that meets all identified Must-Have functional requirements. The system demonstrates a coherent end-to-end workflow: an instructor can create a course, upload materials, assign quizzes and file assignments, and grade student work, while students can enroll, consume content, submit work, receive feedback, and interact with peers through the Social Hub. The automated test suite confirms the correctness of the core workflows, and the SignalR real-time layer provides a modern user experience that distinguishes OzarkLMS from basic CRUD applications.

12.2 Limitations

- Passwords are stored in plain text. This is the most critical security gap and must be addressed before any public or production deployment.
- File storage is local to the server filesystem. This makes horizontal scaling and cloud deployment non-trivial.
- The PDF Toolbox's DOCX conversion only preserves plain text; rich formatting (tables, headers, images) is lost.
- There is no email or SMS notification system; alerts are internal to the application only.
- SignalR hub does not scope broadcasts to room participants; all connected users receive all events.

12.3 Future Work

- Integrate ASP.NET Core Identity or implement bcrypt hashing to secure password storage.
- Migrate file storage to Azure Blob Storage or AWS S3 for scalability and cloud deployment.
- Implement email notifications via SendGrid or SMTP for assignment reminders and grade alerts.
- Scope SignalR hub groups to specific chat rooms to reduce unnecessary broadcast traffic.
- Develop a native mobile wrapper (React Native or .NET MAUI) for iOS and Android.
- Add an advanced analytics dashboard for instructors showing submission rates, grade distributions, and engagement metrics.

13. References

Microsoft. (2024). ASP.NET Core documentation. Retrieved from https://docs.microsoft.com/aspnet/core

Microsoft. (2024). Entity Framework Core documentation. Retrieved from https://docs.microsoft.com/ef/core

Microsoft. (2024). ASP.NET Core SignalR documentation. Retrieved from https://docs.microsoft.com/aspnet/core/signalr

Npgsql. (2024). Npgsql Entity Framework Core provider for PostgreSQL. Retrieved from https://www.npgsql.org/efcore/

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. Retrieved from https://www.postgresql.org/docs/16/

Tailwind Labs. (2024). Tailwind CSS documentation. Retrieved from https://tailwindcss.com/docs

QuestPDF. (2024). QuestPDF documentation. Retrieved from https://www.questpdf.com/

PdfSharp. (2024). PdfSharp library documentation. Retrieved from http://www.pdfsharp.net/

Lucide Contributors. (2024). Lucide Icons. Retrieved from https://lucide.dev/

xUnit.net. (2024). xUnit documentation. Retrieved from https://xunit.net/

Moq. (2024). Moq library documentation. Retrieved from https://github.com/moq/moq4

Appendix A: Testing Documentation

The full Testing Documentation for OzarkLMS is provided as a separate companion document titled:

OzarkLMS — Testing Documentation

That document contains:

- Full Testing Strategy (Unit, Integration, System levels)
- Complete test case specifications for all test cases executed
- Requirements Traceability Matrix (FR-1 through FR-10, NFR-1 through NFR-5)
- Test execution metrics and pass/fail results
- Defect tracking log with severity ratings and resolution notes
- Known limitations and future testing recommendations

The testing documentation should be read in conjunction with this final report to obtain a complete picture of the quality assurance process performed on OzarkLMS.